



LOCAL DEVELOPMENT → PUBLIC WEB, WITHOUT FRICTION.

KillTheHost

v1.6.0 – User Manual

PHP-MNGR · DB-3NGIN3 · MAIL-SRVR · STAX-MNGR
NODE-MNGR · SEC-MNGR · LDOMAIN-3NGIN3

Table of Contents

1. Introduction & Overview	2	8. NODE-MNGR — Node.js App Manager	10
2. Installation & Requirements	3	9. SEC-MNGR — Security Manager	12
3. The Launcher — Control Panel	4	10. LDOMAIN-3NGIN3 — Local DNS & Proxy	14
4. PHP-MNGR — PHP Project Manager	5	11. Domain Sync & Cloudflare Tunnels	18
5. DB-3NGIN3 — Database Manager	7	12. Data Locations Reference	19
6. MAIL-SRVR — Self-Hosted Email Server	8	13. Networking & Port Reference	20
7. STAX-MNGR — Docker Stack Manager	9	14. FAQ & Common Issues	21
		15. License & Contributing	23

1. Introduction & Overview

KillTheHost is an all-in-one local development platform that bridges the gap between localhost and the live web. It bundles seven open-source, single-file Python tools into a unified browser-based workflow — letting you run PHP sites, manage databases, host your own email, deploy Docker stacks, manage Node.js apps, lock everything down with 24/7 security monitoring, and give every local project a real domain name with HTTPS — all from one place.

Version 1.5 introduced SEC-MNGR — a security manager and threat dashboard that watches every KillTheHost manager, bans attacking IPs automatically, and monitors the host around the clock.

Version 1.6 introduces LDOMAIN-3NGIN3 — a local domain and reverse proxy manager that gives every local project a real .local-style domain name, HTTPS via a self-signed local CA, and a full HTTP/HTTPS reverse proxy without any external tools.

Tool	Version	Purpose
[L] Launcher	v1.6	Unified browser UI — start, stop, monitor all tools
[PHP] PHP-MNGR	v2.5	Local & public PHP project manager via Docker

[DB] DB-3NGIN3	v1.2	Database service manager — PostgreSQL, MySQL, MariaDB, Redis, MongoDB
[MAIL] MAIL-SRVR	v1.0	Self-hosted email server with full browser mail client
[STAX] STAX-MNGR	v1.0	Docker stack manager — one-click pre-configured stacks
[NODE] NODE-MNGR	v1.0	Node.js app manager — React, Next.js, Vite, Express
[SEC] SEC-MNGR New v1.5	v1.0	Security manager — unified access logs, IP bans, Fail2Ban rules, 24/7 monitoring
[DNS] LDOMAIN-3NGIN3 New v1.6	v1.0	Local domain & reverse proxy manager — DNS, TLS, HTTP/HTTPS proxy, hosts-file adapter

2. Installation & Requirements

System Requirements

Requirement	Notes
Python 3.8+	Standard library only — no pip installs required
Docker	Required for PHP-MNGR, DB-3NGIN3, MAIL-SRVR, STAX-MNGR
Node.js / nvm	Required for NODE-MNGR — install via nvm for version switching
openssl	Required for LDOMAIN-3NGIN3 local CA (pre-installed on most systems)
OS	Linux, macOS, Windows (WSL2 recommended for Docker on Windows)

Install Docker (Ubuntu/Linux)

```
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
# Log out and back in before continuing
```

Install Node.js via nvm (recommended)

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
# Restart terminal, then:
nvm install 20
nvm use 20
```

Clone & Start

```
git clone https://github.com/Official-phdesigns/KillTheHost.git
cd KillTheHost
```

Linux / macOS: `chmod +x launch.sh && ./launch.sh`

Windows: Double-click `launch.bat` or run it from any terminal.

The Launcher opens automatically at `http://localhost:5000`.

Folder Structure

```
KillTheHost/
├─ launch.sh / launch.bat
├─ stop.sh / restart.sh (stop.bat / restart.bat)
├─ LICENSE / README.md
├─ Launcher/
│   └─ launcher.py ← Control panel (port 5000)
│   └─ assets/main/
│       ├── PHP-MNGR v2.5/phpmanager.py
│       ├── DB-3NGIN3 v1.2/db3ngin3.py
│       ├── MAIL-SRVR v1.0/mailserver.py
│       ├── STAX-MNGR v1.0/staxmgr.py
│       ├── NODE-MNGR v1.0/nodemngr.py
│       ├── SEC-MNGR v1.0/sec_mngr.py ← New in v1.5
│       └─ LDOMAIN-3NGIN3 v1.0/ldomain3ngin3.py ← New in v1.6
```

3. The Launcher — Control Panel

The Launcher is a zero-dependency, pure Python browser-based control panel that starts and stops all KillTheHost tools from one place.

Features

- Start / Stop all tools individually or all at once
- Real-time console output per service with live status indicators and uptime timers
- Port monitoring, console filtering by service, and direct links to open each tool in a new tab
- Console output filter buttons for all services including SEC-MNGR and LDOMAIN-3NGIN3

Launcher Port Reference

Service	Port
Launcher UI	5000
PHP-MNGR UI	4280
DB-3NGIN3 UI	7734
MAIL-SRVR UI	6060
STAX-MNGR UI	6161
NODE-MNGR UI	7272
SEC-MNGR UI	8080
LDOMAIN-3NGIN3 UI	8181

To stop the Launcher, press Ctrl+C in the terminal. It will gracefully shut down all running services first.

Stopping & Restarting

Platform	Stop	Restart
Linux / macOS	<code>./stop.sh</code>	<code>./restart.sh</code>
Windows	<code>stop.bat</code>	<code>restart.bat</code>

4. PHP-MNGR — PHP Project Manager

PHP-MNGR lets you spin up, manage, and publish PHP sites using Docker containers — with a full browser-based file manager and inline code editor per site.

Supported PHP Versions

Runs each site inside a `php:VERSION-apache` Docker container:

7.2 · 7.3 · 7.4 · 8.0 · 8.1 · 8.2 · 8.3

Creating a Site

1. Open PHP-MNGR at `http://localhost:4280`, click + **New Site**
2. Enter a site name and PHP version — PHP-MNGR pulls the image and starts the container
3. Your site is available at `http://localhost:810X` (auto-assigned port)

File Manager & Code Editor

- Browse, edit, upload, download, rename, delete, and chmod files
- Inline code editor for PHP, HTML, CSS, and JS · Custom `php.ini` per site

Port Assignments

Service	Port
PHP-MNGR UI	4280
PHP sites (auto-assigned)	8100, 8101, 8102...

Data Locations

What	Where
Site registry	<code>~/.phpmngr/sites.json</code>
Tunnel registry	<code>~/.phpmngr/tunnels.json</code>
Cloudflare credentials	<code>~/.phpmngr/cloudflare.json</code>

Namecheap credentials	<code>~/.phpmngnr/namecheap.json</code>
Site web roots	<code>~/.phpmngnr/sites/<id>/www/</code>
Per-site PHP config	<code>~/.phpmngnr/sites/<id>/php.ini</code>

5. DB-3NGIN3 — Database Manager

DB-3NGIN3 manages PostgreSQL, MySQL, MariaDB, Redis, and MongoDB Docker containers with live status polling, one-click connection strings, and persistent data volumes.

Supported Databases

Database	Versions	Port	User	Password
PostgreSQL	13, 14, 15, 16	5432	postgres	postgres
MySQL	5.7, 8.0, 8.3	3306	admin	admin
MariaDB	10.6, 10.11, 11.3	3307	root	root
Redis	6.2, 7.0, 7.2	6379	—	—
MongoDB	5.0, 6.0, 7.0	27017	admin	admin

Security Notice: Default credentials are for local development only. Do not expose database containers publicly without changing credentials first.

Usage

1. Open DB-3NGIN3 at `http://localhost:7734` , click + **New Instance**
2. Select engine and version — DB-3NGIN3 pulls the image and starts the container
3. Live status polled every 15s · Click connection string button to copy credentials
4. Deleting an instance removes the container but preserves data files on disk

Data Locations

What	Where
Instance metadata	<code>~/.db3ngin3/instances.json</code>
Database files	<code>~/.db3ngin3/data/<instance-id>/</code>

6. MAIL-SRVR — Self-Hosted Email Server

MAIL-SRVR runs a complete email server on your own VPS — handling SMTP delivery, inbound receiving, IMAP inbox access, DKIM signing, SPF and DMARC records, and a full browser-based email client.

Requirements for Live Mode

Requirement	Notes
VPS	Static public IP required. Port 25 must be open outbound.
Domain	Must be managed via Cloudflare DNS (free account works).
Cloudflare token	Shared with PHP-MNGR — enter once in PHP-MNGR Settings.
Recommended VPS	Hetzner, Contabo — port 25 open by default.

Setting Up Email

1. Open MAIL-SRVR at `http://localhost:6060` → Settings → switch to **Live mode**
2. Domain tab: select your Cloudflare zone and confirm your public IP
3. Click **Start Server** — MX, A, SPF, DMARC records provisioned automatically
4. After ~60s click **Provision DKIM** to generate and publish your signing key
5. Add a mailbox in the Accounts tab, then use Compose to send your first email

DNS Records (Auto-Provisioned)

Record	Type	Purpose
mail.yourdomain.com	A (DNS-only)	Mail server address
yourdomain.com	MX	Incoming mail routing
yourdomain.com	TXT (SPF)	Sender Policy Framework
_dmarc.yourdomain.com	TXT (DMARC)	DMARC policy
mail._domainkey.yourdomain.com	TXT (DKIM)	DKIM signing key

Port Assignments

Service	Port
MAIL-SRVR UI	6060
SMTP submission (outbound)	587
IMAP (inbox)	143
Postfix inbound	25
Mailpit UI (dev mode)	8025

7. STAX-MNGR — Docker Stack Manager

STAX-MNGR lets you deploy and manage pre-configured Docker application stacks with a single click. Ships with 10 ready-to-use stacks.

Available Stacks

Stack	Port	Category
VaultWarden	8200	Privacy

Uptime Kuma	3002	Monitoring
Navidrome	4533	Media
Wiki.js	3003	Productivity
Jellyfin	8096	Media
Ollama	11434	AI
Open WebUI	3000	AI
WordPress	8080	CMS
Nextcloud	8081	Productivity
Gitea	3001	Development

Usage

- Open STAX-MNGR at `http://localhost:6161` and click **Deploy** on any stack
- STAX-MNGR pulls the Docker Compose setup and starts all containers
- Each stack gets its own card with live status and controls
- Persistent data volumes survive container restarts

8. NODE-MNGR — Node.js App Manager

NODE-MNGR is a full browser-based Node.js app manager for React, Next.js, Vite, Express, and any npm/yarn project.

Supported Frameworks

React · Next.js · Vite · Express · Node.js (plain) · any npm or yarn project

Dashboard Overview

Panel	What it shows
Apps	All deployed apps with live status, port, package manager, and controls

Logs	Real-time log output per app
System	Host resource usage, nvm environment, and Node.js version info

Deploying an App

1. Open NODE-MNGR at `http://localhost:7272`, click + **Deploy App**
2. Point it at your local project folder (or paste a GitHub repo URL)
3. Select package manager (npm or yarn) and Node.js version
4. NODE-MNGR installs dependencies and starts your app automatically
5. Click **Open** on the app card to view it in the browser
6. Click **Cloudflare Tunnel** on the app card to assign a public domain

App Card Controls

Control	Action
Stop	Gracefully stop the running app process
Restart	Restart the app (re-runs the start command)
Open	Open the app in a new browser tab
Node	Switch the Node.js version for this app via nvm
Settings	App settings — rename, change start command, delete

Node.js Version Switching

NODE-MNGR uses nvm to manage Node.js versions per app. Install nvm:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
```

Ports & Data

What	Value
NODE-MNGR UI Port	7272
Apps (auto-assigned)	3100, 3101, 3102...

App registry	<code>~/.nodemngr/apps.json</code>
--------------	------------------------------------

App data & builds	<code>~/.nodemngr/apps/<id>/</code>
-------------------	---

9. SEC-MNGR — Security Manager New in v1.5

SEC-MNGR is the security layer for the whole KillTheHost stack. It is a single Python 3.8+ file (stdlib-only) with an embedded browser dashboard. Start it from the launcher, or run directly:

```
python3 "Launcher/assets/main/SEC-MNGR v1.0/sec_mngr.py"
# → http://127.0.0.1:8080

python3 "Launcher/assets/main/SEC-MNGR v1.0/sec_mngr.py" \
    --host 127.0.0.1 --port 8080 --no-browser
```

By default the panel binds to 127.0.0.1 — it has no login. For remote access use an SSH tunnel: `ssh -L 8080:127.0.0.1:8080 user@vps`

Dashboard Tabs

Tab	What it shows
Overview	Threat level, posture score, service health, 24h traffic chart, event timeline
Access Logs	Unified access log — search, filter, export (CSV/JSON)
IP Manager	IP intelligence, manual bans, allowlist, firewall export
Detection Rules	Fail2Ban-style rules — thresholds, windows, durations
Monitoring	Service UP/DOWN, CPU/mem/disk, Docker, UFW, port scan
System Health	Latest health snapshot and log sources
Settings	Retention, intervals, detection mode, known ports

What SEC-MNGR Watches

SEC-MNGR tails the log files of every KillTheHost manager: `~/.phpmngr/`, `~/.db3nginx3/`, `~/.mailsrvr/`, `~/.staxmngr/`, `~/.nodemngr/`, `~/.killthehost/launcher.log`, and (from v1.6) `~/.ldomain3nginx3/access_logs.jsonl`. It also reads Docker logs from manager containers.

Securing Your Stack with SEC-MNGR

1. Start SEC-MNGR from the launcher and open `http://127.0.0.1:8080`
2. In **IP Manager**, add your own IP to the Allowlist first — so you never lock yourself out
3. Review **Detection Rules** thresholds; set Settings → detection mode to *monitor* to only log while tuning
4. Ban an IP or CIDR range from **IP Manager** → **Ban an IP / CIDR**
5. For real firewall bans, run with root or passwordless sudo and have UFW/iptables installed
6. Use **Access Logs** to search traffic across every service and export to CSV/JSON

Detection Rules (Defaults)

Rule	Trigger	Ban
AUTH_FAIL	5 auth failures in 10 min	1 h
RATE_FLOOD	100 HTTP requests in 60 s	30 min
PATH_PROBE	10 × 404 in 5 min	2 h
MALFORMED	20 malformed requests in 10 min	1 h
MAIL_FAIL	3 SMTP/IMAP auth failures in 5 min	4 h
DB_FAIL	5 database auth failures in 10 min	2 h
REPEAT_OFFENDER	IP banned 3 times	permanent

Enforcement Model

1. UFW (`ufw prepend deny from <ip>`) if installed and active
2. iptables / ip6tables (plus DOCKER-USER chain) otherwise
3. HTTP-level blocking always applies — banned IPs receive **403**

Firewall commands need root or passwordless `sudo -n`. Without them, SEC-MNGR does not fail — it logs the exact commands to run and relies on HTTP-level blocking.

Data Locations

Item	Path
Access log (JSONL)	<code>~/.secmgr/access_logs.jsonl</code>
Security events	<code>~/.secmgr/events.jsonl</code>
Bans, allowlist	<code>~/.secmgr/bans.json</code>
Detection rules	<code>~/.secmgr/rules.json</code>
Settings	<code>~/.secmgr/settings.json</code>
Health snapshot	<code>~/.secmgr/health.json</code>
Per-IP statistics	<code>~/.secmgr/ip_stats.json</code>
Blocklist (one IP/CIDR per line)	<code>~/.secmgr/blocklist.txt</code>

10. LDOMAIN-3NGIN3 — Local Domain & Reverse Proxy Manager New in v1.6

LDOMAIN-3NGIN3 gives every local project a real domain name (e.g. `app.dev.local`), HTTPS via a self-signed local CA, and a full HTTP/HTTPS reverse proxy — all without touching a public DNS provider, installing nginx, or running external tools. It is a single Python 3.8+ file (stdlib only) with an embedded browser dashboard.

Start it from the launcher, or run directly:

```
python3 "Launcher/assets/main/LDOMAIN-3NGIN3 v1.0/ldomain3nginx.py"
# → http://localhost:8181

python3 "Launcher/assets/main/LDOMAIN-3NGIN3 v1.0/ldomain3nginx.py" \
    --port 8181 --no-browser
```

Binding the reverse proxy to :80 and :443 requires root or passwordless sudo. Without it, LDOMAIN-3NGIN3 falls back to :8180 (HTTP) and :8143 (HTTPS) automatically.

How It Works

```
Browser / curl → https://app.dev.local
                |
                | LDOMAIN-3NGIN3 (:443 / :8143)
                | TLS termination
                |
                | reverse proxy → banned? → 403 (SEC-MNGR blocklist)
                |
                | http://localhost:4280 (your PHP site, Node.js app, etc.)

LAN device → DNS query → LDOMAIN-3NGIN3 (:5353) → A record response
hosts file → 127.0.0.1 (single-machine, no DNS server needed)
```

Features

Feature	Details
Local DNS resolver	Pure-Python UDP DNS on :5353 — A-record responses for registered domains
Reverse proxy	HTTP :80/8180 and HTTPS :443/8143 — raw TCP sockets, WebSocket-capable
TLS via local CA	Root CA + per-domain certs with SAN extensions, generated with openssl
Hosts-file adapter	Writes 127.0.0.1 entries to /etc/hosts inside managed block markers
SEC-MNGR integration	Writes access logs in SEC-MNGR JSONL format; enforces SEC-MNGR IP bans before forwarding
Per-domain health checks	Background TCP port checks every 60 s — UP/DOWN status in dashboard
Integration API	POST /api/integrate — other managers can register/remove domains programmatically
Browser dashboard	Dark-theme UI at http://localhost:8181

Setting Up a Local Domain

1. Start LDOMAIN-3NGIN3 from the launcher and open `http://localhost:8181`
2. Click + **Add Domain** and enter a domain name (e.g. `myapp.dev.local`) and the upstream host/port (e.g. `localhost:4280`)
3. Enable **TLS** to get HTTPS; enable **Hosts File** to write to `/etc/hosts`
4. Install the root CA once in your browser or OS (see below)
5. Open `https://myapp.dev.local` in your browser — it routes to `localhost:4280` with a trusted certificate

CA Installation (One-Time Per Device)

Linux (system-wide)

```
sudo cp ~/.ldomain3nginx3/ca/ca.crt /usr/local/share/ca-certificates/ldomain3nginx3.crt
sudo update-ca-certificates
```

macOS

```
sudo security add-trusted-cert -d -r trustRoot \
-k /Library/Keychains/System.keychain ~/.ldomain3nginx3/ca/ca.crt
```

Windows

```
certutil -addstore -f "ROOT" "$env:USERPROFILE\.ldomain3nginx3\ca\ca.crt"
```

Firefox

Preferences → Privacy & Security → View Certificates → Authorities → Import → select `ca.crt` → check "Trust this CA to identify websites".

iOS

AirDrop or email `ca.crt` → Settings → General → VPN & Device Management → install → Settings → General → About → Certificate Trust Settings → enable.

Android

Settings → Security → Install from storage → select `ca.crt` .

Port Assignments

Service	Port	Fallback
LDOMAIN-3NGIN3 management UI	8181	—
HTTP reverse proxy	80	8180 (if :80 unavailable)
HTTPS reverse proxy	443	8143 (if :443 unavailable)
LAN DNS resolver	5353 (UDP)	—

SEC-MNGR Integration

LDOMAIN-3NGIN3 automatically integrates with SEC-MNGR with no configuration required:

- **Access logs:** Every proxied request is logged to `~/ldomain3nginx/access_logs.jsonl` in SEC-MNGR's unified JSONL format (`ts` , `timestamp` , `ip` , `method` , `path` , `status` , `user_agent` , `service` , `event_type` , `source` , `detail`). SEC-MNGR picks these up automatically from v1.6 onwards.
- **Blocklist enforcement:** LDOMAIN-3NGIN3 reads `~/secmngr/blocklist.txt` every 30 seconds. Any IP on the blocklist receives a 403 Forbidden before the request is forwarded to the upstream.

REST API

Method	Endpoint	Description
GET	<code>/api/domains</code>	List all registered domains with health status
POST	<code>/api/domains</code>	Add a domain
DELETE	<code>/api/domains/{domain}</code>	Remove a domain and clean up certs + hosts entries
GET	<code>/api/health</code>	Per-domain health check results
GET	<code>/api/logs?limit=100</code>	Recent access log entries
POST	<code>/api/integrate</code>	Integration API for other managers
GET	<code>/api/ca</code>	CA certificate info and install instructions

Data Locations

Item	Path
Domain registry	<code>~/.ldomain3nginx3/registry.json</code>
Access log (JSONL, SEC-MNGR format)	<code>~/.ldomain3nginx3/access_logs.jsonl</code>
Root CA key	<code>~/.ldomain3nginx3/ca/ca.key</code>
Root CA certificate	<code>~/.ldomain3nginx3/ca/ca.crt</code>
Per-domain certificates	<code>~/.ldomain3nginx3/certs/<domain>.crt</code>
Per-domain private keys	<code>~/.ldomain3nginx3/certs/<domain>.key</code>

11. Domain Sync & Cloudflare Tunnels

KillTheHost connects your local services to real domains via Namecheap DNS sync and Cloudflare Tunnels — no port forwarding, no router config, works on CGNAT connections.

Connecting a Domain

1. Whitelist your public IP in Namecheap API settings
2. Create a scoped Cloudflare API token with: Account → Cloudflare Tunnel (Edit & Read), Zone → Zone (Read), Zone → DNS (Edit & Read)
3. Open PHP-MNGR → Settings → Cloudflare, paste your API token and save
4. Open PHP-MNGR → Settings → Namecheap, enter your API key and username
5. Go to Domains & Tunnels, select a domain, map it to your local service, and apply

Going Live with Cloudflare

1. Go to Domains & Tunnels and click **Tunnel Site** on any domain
2. Select your PHP site from the dropdown and hit **Tunnel Now**
3. Your site is now reachable at your real domain

Cloudflare Tunnels work on CGNAT connections (e.g. T-Mobile 5G home internet) where traditional port forwarding is impossible.

12. Data Locations Reference

Tool	What	Where
PHP-MNGR	Site registry	<code>~/.phpmngr/sites.json</code>
PHP-MNGR	Site web roots	<code>~/.phpmngr/sites/<id>/www/</code>
PHP-MNGR	Cloudflare credentials	<code>~/.phpmngr/cloudflare.json</code>
PHP-MNGR	Namecheap credentials	<code>~/.phpmngr/namecheap.json</code>
DB-3NGIN3	Instance metadata	<code>~/.db3ngin3/instances.json</code>
DB-3NGIN3	Database files	<code>~/.db3ngin3/data/<instance-id>/</code>
MAIL-SRVR	Configuration	<code>~/.mailsrvr/config.json</code>
MAIL-SRVR	Accounts	<code>~/.mailsrvr/accounts.json</code>
MAIL-SRVR	Mail storage	<code>~/.mailsrvr/mail-data/</code>
MAIL-SRVR	DKIM keys & Postfix config	<code>~/.mailsrvr/config/</code>
NODE-MNGR	App registry	<code>~/.nodemngr/apps.json</code>
NODE-MNGR	App data & builds	<code>~/.nodemngr/apps/<id>/</code>
SEC-MNGR	Access log (JSONL)	<code>~/.secmngr/access_logs.jsonl</code>
SEC-MNGR	Security events	<code>~/.secmngr/events.jsonl</code>
SEC-MNGR	Bans, allowlist	<code>~/.secmngr/bans.json</code>
SEC-MNGR	Detection rules	<code>~/.secmngr/rules.json</code>
SEC-MNGR	Settings	<code>~/.secmngr/settings.json</code>
SEC-MNGR	Blocklist	<code>~/.secmngr/blocklist.txt</code>
LDOMAIN-3NGIN3	Domain registry	<code>~/.ldomain3ngin3/registry.json</code>

LDOMAIN-3NGIN3	Access log (JSONL)	<code>~/.ldomain3nginx3/access_logs.jsonl</code>
LDOMAIN-3NGIN3	Root CA certificate	<code>~/.ldomain3nginx3/ca/ca.crt</code>
LDOMAIN-3NGIN3	Per-domain certs	<code>~/.ldomain3nginx3/certs/<domain>.crt</code>
Launcher	Event log	<code>~/.killthehost/launcher.log</code>

13. Networking & Port Reference

Service	Port	Notes
Launcher UI	5000	Main control panel
PHP-MNGR UI	4280	PHP project manager
PHP sites	8100+	Auto-assigned per site
DB-3NGIN3 UI	7734	Database manager
PostgreSQL	5432	Default DB port
MySQL	3306	Default DB port
MariaDB	3307	Default DB port
Redis	6379	Default DB port
MongoDB	27017	Default DB port
MAIL-SRVR UI	6060	Email server & client
SMTP submission	587	Outbound email
IMAP	143	Inbox access
Postfix inbound	25	Incoming email (VPS only)
Mailpit UI (dev)	8025	Dev mode email preview
STAX-MNGR UI	6161	Docker stack manager
NODE-MNGR UI	7272	Node.js app manager
Node.js apps	3100+	Auto-assigned per app

SEC-MNGR UI	8080	Security manager (127.0.0.1 only)
LDOMAIN-3NGIN3 UI	8181	Local domain manager
HTTP reverse proxy	80 / 8180	LDOMAIN-3NGIN3 (8180 if :80 unavailable)
HTTPS reverse proxy	443 / 8143	LDOMAIN-3NGIN3 (8143 if :443 unavailable)
LAN DNS resolver	5353 (UDP)	LDOMAIN-3NGIN3 DNS server

14. FAQ & Common Issues

Docker permission denied on Linux

```
sudo usermod -aG docker $USER
# Log out and back in
```

Port already in use

```
lsof -i :PORT
kill -9 PID
```

Cloudflare tunnel not connecting

- Ensure your Cloudflare API token has correct permissions (Tunnel Edit, DNS Edit, Zone Read)
- Check that `cloudflared` is installed and in your PATH
- Verify the domain is active in your Cloudflare account

Email not delivering (Live mode)

- Confirm outbound port 25 is open on your VPS
- Run the deliverability checklist in MAIL-SRVR → Settings
- Ensure DKIM has been provisioned (wait 60s after starting the server)
- Check SPF and DMARC records are published in Cloudflare DNS

NODE-MNGR app won't start

- Ensure nvm is installed and accessible in your shell — run `nvm ls` to check versions
- Verify your project has a valid `package.json` with a start script
- Check the Logs tab in NODE-MNGR for the full error output

LDOMAIN-3NGIN3: browser shows "Your connection is not private"

- The root CA has not been installed in this browser/OS yet. Follow the CA Installation steps in Chapter 10.
- After installing, restart the browser and try again.

LDOMAIN-3NGIN3: reverse proxy returns 502 Bad Gateway

- The upstream service (e.g. `localhost:4280`) is not running. Start it from the Launcher first.
- Check the health status in the LDOMAIN-3NGIN3 dashboard — a red indicator means the upstream is down.

LDOMAIN-3NGIN3: DNS not resolving on LAN device

- On the LAN device, set the DNS server to your machine's LAN IP (e.g. `192.168.1.x`) port 5353.
- Alternatively, enable the Hosts File option for single-machine resolution without a DNS server.

LDOMAIN-3NGIN3: :80 or :443 already in use

LDOMAIN-3NGIN3 automatically falls back to :8180 and :8143 when :80/:443 are unavailable. Use those ports in your browser, or stop whatever is holding :80/:443 first.

Services don't restart after reboot

```
docker update --restart unless-stopped <site-name>
```

systemd service (Linux):

```
# Save as ~/.config/systemd/user/killthehost.service
[Unit]
Description=KillTheHost Launcher

[Service]
ExecStart=/bin/sh /path/to/KillTheHost/launch.sh
WorkingDirectory=/path/to/KillTheHost
Restart=on-failure

[Install]
WantedBy=default.target
```

```
systemctl --user enable --now killthehost
```

15. License & Contributing

License

KillTheHost is licensed under the **GNU Affero General Public License v3.0 (AGPL-3.0)**.

Any modified versions of this software that are run over a network must also be made available as open source under the same license.

Contributing

```
git clone https://github.com/Official-phdesigns/KillTheHost.git
cd KillTheHost
git checkout -b feature/your-feature-name
```

- Make your changes and write or update tests if applicable
- Open a pull request with a clear description
- For major changes, open an issue first to discuss what you'd like to change

Roadmap

Feature	Status
---------	--------

PHP site management (PHP-MNGR)	✓ Shipped
Database service management (DB-3NGIN3)	✓ Shipped
Cloudflare tunnel integration	✓ Shipped
Namecheap domain sync	✓ Shipped
Unified cross-platform launcher (v1.1)	✓ Shipped
Self-hosted email server (MAIL-SRVR v1.1)	✓ Shipped
Docker stack manager (STAX-MNGR v1.0)	✓ Shipped
Node.js app manager (NODE-MNGR v1.1)	✓ Shipped — v1.4
Additional domain registrar support	✓ Shipped
Security manager with Fail2Ban-style detection (SEC-MNGR v1.0)	✓ Shipped — v1.5
Local domain & reverse proxy manager (LDOMAIN-3NGIN3 v1.0)	✓ Shipped — v1.6
Multi-domain email support in MAIL-SRVR	→ Planned
MAIL-SRVR relay/smarthost option	→ Planned
NODE-MNGR: GitHub auto-deploy / webhook triggers	→ Planned
NODE-MNGR: Environment variable manager per app	→ Planned